

100 JavaScript

Interview Questions & Answers

Concise · Interview-Ready · With Examples

Total Questions	100
Topics Covered	Fundamentals, Async, ES6+, DOM, Performance
Style	Concise • Interview-Ready • With Code Examples

SECTION 1: JavaScript Fundamentals

Q1. What is JavaScript?

JavaScript is a lightweight, interpreted, single-threaded, dynamically typed programming language primarily used to make web pages interactive. It runs in the browser and on the server via Node.js, and supports object-oriented, functional, and event-driven programming paradigms.

Q2. What are the data types in JavaScript?

Data types define the kind of value a variable can hold. JavaScript has two categories:

Primitive — immutable, stored by value: String, Number, BigInt, Boolean, undefined, null, Symbol.

Non-Primitive — mutable, stored by reference: Object, Array, Function, Date.

```
let name    = 'Sanket';    // String
let age     = 23;          // Number
let active  = true;        // Boolean
let x       = undefined;   // undefined
let y       = null;        // null
let arr     = [1, 2, 3];   // Array (Object)
let obj     = { a: 1 };    // Object
```

Q3. What is the difference between null, NaN, and undefined?

undefined — variable declared but never assigned a value.

null — intentionally assigned to represent 'no value' or 'empty'.

NaN — result of an invalid or failed numeric operation. `typeof NaN === 'number'`.

```
let a;           // undefined
let b = null;     // null – explicitly empty
let c = 'abc' * 2; // NaN – invalid math
console.log(typeof a); // 'undefined'
console.log(typeof b); // 'object' (JS quirk)
console.log(isNaN(c)); // true
```

Q4. Explain the concept of hoisting in JavaScript.

Hoisting is JavaScript's behaviour of moving variable and function declarations to the top of their scope before execution.

var → hoisted and initialized as undefined. **let/const** → hoisted but stay in the Temporal Dead Zone (TDZ) — accessing them before declaration throws ReferenceError. **Function declarations** → fully hoisted and callable before their definition.

```
console.log(a); // undefined (var hoisted)
var a = 5;
console.log(b); // ReferenceError (TDZ)
let b = 10;
greet();        // Works! – function fully hoisted
function greet() { console.log('Hello'); }
```

Q5. What is a closure in JavaScript?

A closure is when an inner function remembers and accesses variables from its outer function's scope, even after the outer function has finished executing. Think of it as a function carrying a backpack of the variables it was created with.

```
function counter() {  
  let count = 0;  
  return function() {  
    count++;  
    return count;  
  };  
}  
const increment = counter();  
increment(); // 1  
increment(); // 2 - count is still remembered
```

Q6. What is the difference between == and === in JavaScript?

== (loose equality) — compares values after type coercion; JS converts types to match before comparing.

=== (strict equality) — compares both value AND type with no conversion. Always prefer === to avoid unexpected bugs.

```
0 == '0'           // true - '0' coerced to number  
0 === '0'          // false - number vs string  
null == undefined  // true (loose)  
null === undefined // false (strict)  
false == 0         // true (coercion)  
false === 0        // false
```

Q7. Explain prototypal inheritance in JavaScript.

Objects inherit properties and methods from other objects via the prototype chain. Every object has an internal `[[Prototype]]` link. When a property is not found on the object itself, JavaScript walks up the chain until it reaches null.

```
const animal = { breathes: true };  
const dog    = Object.create(animal);  
dog.name     = 'Rex';  
console.log(dog.breathes); // true - inherited  
console.log(dog.name);     // 'Rex' - own property
```

Q8. What are the different ways to define a function in JavaScript?

Function Declaration — hoisted, callable before definition.

Function Expression — not hoisted, assigned to a variable.

Arrow Function — concise, no own 'this'.

IIFE — runs immediately upon definition.

Method Shorthand — inside objects.

Constructor Function / new Function() — dynamic, rarely used.

```
function greet() { return 'Hi'; } // Declaration
const greet = function() { return 'Hi'; } // Expression
const greet = () => 'Hi'; // Arrow
(function() { console.log('IIFE'); })(); // IIFE
const obj = { greet() { return 'Hi'; } }; // Method
```

Q9. How does event delegation work in JavaScript?

Instead of attaching a listener to each child element, you attach one listener to a parent. It relies on event bubbling — events triggered on children bubble up to the parent, where you use `event.target` to identify which child triggered it. This is more memory-efficient and works for dynamically added elements.

```
document.getElementById('list')
  .addEventListener('click', function(e) {
    if (e.target.tagName === 'LI') {
      console.log('Clicked:', e.target.textContent);
    }
  });
```

Q10. What is the purpose of the 'this' keyword in JavaScript?

'this' refers to the object that is currently executing the function. Its value depends on how the function is called:

Context	this refers to
Global scope	window (browser) / global (Node)
Object method	The object itself
Arrow function	Lexically inherited from outer scope
call / apply / bind	Explicitly set object
Constructor (new)	Newly created instance

```
const obj = {
  name: 'Sanket',
  greet() { console.log(this.name); } // 'Sanket'
};
const arrow = () => console.log(this); // outer this
```

SECTION 2: Objects & Functions

Q11. What are the different ways to create objects in JavaScript?

Object Literal — simplest and most common way.

Object.create() — creates object with specified prototype.

Constructor Function — used with 'new' keyword.

ES6 Class — syntactic sugar over constructor functions.

Factory Function — returns a new object from a function.

```
const obj1 = { name: 'Sanket' };           // Literal
const obj2 = Object.create(obj1);          // create()
function Person(n) { this.name = n; }      // Constructor
const obj3 = new Person('Sanket');
class Car { constructor(m) { this.model = m; } } // Class
const obj4 = new Car('Tesla');
```

Q12. Explain the concept of callback functions in JavaScript.

A callback is a function passed as an argument to another function, to be executed after an operation completes. It is the foundation of asynchronous JavaScript.

```
function fetchData(callback) {
  setTimeout(() => {
    callback('Data loaded');
  }, 1000);
}
fetchData((result) => {
  console.log(result); // 'Data loaded' after 1s
});
```

Q14. What is the purpose of the 'bind' method in JavaScript?

bind() creates a new function with 'this' permanently set to the specified object, and optionally pre-fills arguments (partial application). Unlike call/apply, it does not invoke the function immediately.

```
const person = { name: 'Sanket' };
function greet(greeting) {
  console.log(greeting + ', ' + this.name);
}
const boundGreet = greet.bind(person, 'Hello');
boundGreet(); // 'Hello, Sanket'
```

Q16. What is the 'typeof' operator used for?

typeof returns a string indicating the data type of its operand. It is commonly used for type checking at runtime.

```
typeof 'hello'      // 'string'
typeof 42           // 'number'
typeof true         // 'boolean'
typeof undefined    // 'undefined'
typeof null         // 'object' (known JS bug)
typeof {}           // 'object'
typeof []           // 'object'
typeof function(){} // 'function'
typeof Symbol()     // 'symbol'
```

Q45. What is the purpose of the 'arguments' object in JavaScript?

The 'arguments' object is an array-like object available inside regular (non-arrow) functions that contains all passed arguments, regardless of how many parameters are defined. Arrow functions do NOT have their own arguments object.

```
function sum() {
  let total = 0;
  for (let i = 0; i < arguments.length; i++) {
    total += arguments[i];
  }
  return total;
}
sum(1, 2, 3); // 6
// Modern alternative: rest params
const sum2 = (...args) => args.reduce((a,b) => a+b, 0);
```

Q46. What are the different ways to define methods in JavaScript objects?

Methods can be defined using: regular function properties, method shorthand (ES6), arrow functions (no own 'this'), and prototype assignment.

```
const obj = {
  greet: function() { return 'Hi'; }, // traditional
  hello() { return 'Hello'; },        // shorthand (ES6)
  arrow: () => 'Arrow',                // no own 'this'
};
function Person(name) { this.name = name; }
Person.prototype.greet = function() { // prototype
  return 'Hi, ' + this.name;
};
```

Q49. What is the purpose of the 'apply' and 'call' methods in JavaScript?

Both call() and apply() invoke a function immediately with a specified 'this'. The difference is how arguments are passed: call() takes arguments individually; apply() takes them as an array.

```
function greet(greeting, punct) {
  console.log(greeting + ' ' + this.name + punct);
}
const user = { name: 'Sanket' };
greet.call(user, 'Hello', '!'); // Hello Sanket!
greet.apply(user, ['Hi', '?']); // Hi Sanket?
```

Q87. How do you handle inheritance in JavaScript?

JavaScript supports inheritance via prototype chains, ES6 classes (using extends), and Object.create(). Classes are syntactic sugar over prototypal inheritance.

```

class Animal {
  constructor(name) { this.name = name; }
  speak() { return this.name + ' makes a sound.'; }
}
class Dog extends Animal {
  speak() { return this.name + ' barks.'; }
}
const d = new Dog('Rex');
d.speak(); // 'Rex barks.'

```

SECTION 3: Scope, Closures & Memory

Q30. What is the difference between let, const, and var in JavaScript?

Feature	var	let	const
Scope	Function	Block	Block
Hoisting	Yes (undefined)	Yes (TDZ)	Yes (TDZ)
Re-declarable	Yes	No	No
Re-assignable	Yes	Yes	No
Global object prop	Yes	No	No

Q36. What are the differences between let and var?

var — function-scoped, hoisted as undefined, can be re-declared, becomes a property of the global object.

let — block-scoped, hoisted but in TDZ (not accessible before declaration), cannot be re-declared in the same scope.

```

var x = 1;
if (true) { var x = 2; }
console.log(x); // 2 – same variable!
let y = 1;
if (true) { let y = 2; }
console.log(y); // 1 – block scoped

```

Q40. How does JavaScript handle variable scoping?

JavaScript uses three levels of scope: **Global** — accessible everywhere. **Function** — variables declared inside a function, accessible only within it. **Block** — let/const declared inside {}, accessible only within that block. Scope chain means inner scopes can access outer scope variables, but not vice versa.

```

let global = 'I am global';
function outer() {
  let outerVar = 'outer';
  function inner() {
    let innerVar = 'inner';
    console.log(outerVar); // accessible
    console.log(global);  // accessible
  }
  // console.log(innerVar); // ReferenceError
}

```

Q47. Explain the concept of memoization and its benefits.

Memoization is an optimization technique where the results of expensive function calls are cached. When the same inputs occur again, the cached result is returned instead of recomputing. Benefits: reduces redundant computation, improves performance for pure functions.

```

function memoize(fn) {
  const cache = {};
  return function(...args) {
    const key = JSON.stringify(args);
    if (cache[key]) return cache[key];
    cache[key] = fn(...args);
    return cache[key];
  };
}
const factorial = memoize(n => n <= 1 ? 1 : n * factorial(n-1));
factorial(5); // computed
factorial(5); // returned from cache

```

Q95. What are the benefits of using closures in JavaScript?

Closures enable: **Data privacy** — variables are hidden from the outside scope. **Stateful functions** — functions remember state between calls. **Partial application / currying** — pre-fill arguments. **Module pattern** — encapsulate logic and expose only what's needed.

```

function makeCounter() {
  let count = 0;          // private variable
  return {
    increment: () => ++count,
    decrement: () => --count,
    value:    () => count,
  };
}
const c = makeCounter();
c.increment(); // 1
c.increment(); // 2
c.value();    // 2

```

Q26. Explain the concept of currying in JavaScript.

Currying transforms a function that takes multiple arguments into a series of functions that each take one argument. It enables partial application and function composition.

```
// Normal function
function add(a, b) { return a + b; }
// Curried version
function curriedAdd(a) {
  return function(b) {
    return a + b;
  };
}
const add5 = curriedAdd(5);
add5(3); // 8
add5(10); // 15
```

Q21. How does JavaScript handle memory management?

JavaScript uses automatic memory management via a Garbage Collector (GC). Memory is allocated when variables/objects are created and freed when they are no longer reachable (no references pointing to them). The main algorithm used is Mark-and-Sweep — it marks all reachable objects from roots (global, stack) and sweeps/frees the rest.

```
let obj = { name: 'Sanket' }; // allocated
obj = null; // reference removed – eligible for GC
// Memory leak example – uncleared interval
const id = setInterval(() => {}, 1000);
clearInterval(id); // always clean up!
```

Q83. How does JavaScript's garbage collection work?

JavaScript's GC uses the Mark-and-Sweep algorithm. Starting from roots (global object, call stack), it marks all reachable objects. Unreachable objects are swept and their memory freed. Modern engines (V8) use generational GC — short-lived objects are collected in the 'young generation'; long-lived ones are promoted to 'old generation' and collected less frequently.

SECTION 4: Asynchronous JavaScript & Event Loop

Q15. Explain the concept of AJAX in JavaScript.

AJAX (Asynchronous JavaScript and XML) allows web pages to send/receive data from a server in the background without reloading the page. Today it uses JSON rather than XML, and is implemented via XMLHttpRequest or the Fetch API.

```
// Modern AJAX with Fetch
fetch('https://api.example.com/data')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Q17. How does JavaScript handle errors and exceptions?

JavaScript uses try...catch...finally for synchronous error handling. Errors can be thrown manually with throw. For async code, use .catch() on Promises or try/catch inside async/await functions.

```
try {
  JSON.parse('invalid json');
} catch (err) {
  console.error(err.message); // caught!
} finally {
  console.log('always runs');
}
// Custom error
throw new Error('Something went wrong');
```

Q19. What is the purpose of async and await keywords?

async/await is syntactic sugar over Promises that makes asynchronous code look and behave like synchronous code. An async function always returns a Promise. await pauses execution inside the function until the Promise resolves, without blocking the main thread.

```
async function getUser() {
  try {
    const res = await fetch('/api/user');
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}
getUser();
```

Q22. Explain the concept of the event loop in JavaScript.

The event loop is what allows JavaScript (single-threaded) to handle asynchronous operations. It continuously checks: if the Call Stack is empty, it picks the next task from the Callback Queue (macrotasks) or Microtask Queue (Promises/queueMicrotask) — microtasks always run before the next macrotask.

```
console.log('1 - start');
setTimeout(() => console.log('3 - timeout'), 0);
Promise.resolve().then(() => console.log('2 - promise'));
console.log('4 - end');
// Output: 1, 4, 2, 3
```

Q24. What is a promise in JavaScript?

A Promise is an object representing the eventual completion or failure of an asynchronous operation. It has three states: **Pending** — initial state. **Fulfilled** — operation completed successfully. **Rejected** — operation failed.

```
const promise = new Promise((resolve, reject) => {
  const success = true;
  if (success) resolve('Done!');
  else reject('Failed!');
});
promise
  .then(result => console.log(result)) // 'Done!'
  .catch(err => console.error(err));
```

Q25. How do you handle errors in promises?

Use `.catch()` at the end of a Promise chain to catch any rejection. In `async/await`, use `try/catch`. `Promise.allSettled()` is useful when you want all results regardless of failure.

```
// .catch()
fetch('/api')
  .then(r => r.json())
  .catch(err => console.error('Error:', err));
// async/await try-catch
async function load() {
  try { const r = await fetch('/api'); }
  catch (e) { console.error(e); }
}
```

Q34. How do you handle asynchronous operations in JavaScript?

Three main approaches: **Callbacks** — oldest, can lead to callback hell. **Promises** — chainable, better error handling. **async/await** — cleanest, reads like synchronous code. For parallel operations use `Promise.all()`; for any-first use `Promise.race()`.

```
// Parallel with Promise.all
const [user, posts] = await Promise.all([
  fetch('/api/user').then(r => r.json()),
  fetch('/api/posts').then(r => r.json()),
]);
```

Q50. Explain the event loop and how it handles async operations.

The Call Stack executes synchronous code. Web APIs (`setTimeout`, `fetch`) handle async tasks off-thread. When complete, their callbacks go to the Callback Queue. The Event Loop moves callbacks to the Call Stack only when it is empty. Microtasks (Promises) have higher priority than macrotasks (`setTimeout`).

```
// Execution order
console.log('sync 1');
setTimeout(() => console.log('macro'), 0);
Promise.resolve().then(() => console.log('micro'));
console.log('sync 2');
// Output: sync 1, sync 2, micro, macro
```

Q56. Explain the difference between synchronous and asynchronous JavaScript.

Synchronous — code executes line by line; each line waits for the previous to finish. Blocks the thread.

Asynchronous — operations run in the background; the main thread continues without waiting. Results are handled via callbacks, promises, or `async/await`.

```
// Synchronous – blocks
const data = fs.readFileSync('file.txt'); // waits
// Asynchronous – non-blocking
fs.readFile('file.txt', (err, data) => {
  console.log(data); // runs when ready
});
console.log('This runs before file is read');
```

Q88. What is the difference between synchronous and asynchronous functions?

Synchronous functions — execute completely before returning, blocking further execution. **Asynchronous functions** — return immediately (often a Promise) and complete later, allowing other code to run in the meantime.

```
// Sync – returns value directly
function addSync(a, b) { return a + b; }
// Async – returns a Promise
async function addAsync(a, b) {
  return a + b; // wraps result in Promise
}
addAsync(1, 2).then(console.log); // 3
```

Q89. How does the async keyword work with functions?

Marking a function with async makes it always return a Promise. Inside it, you can use await to pause execution until a Promise resolves. Errors thrown inside are caught by .catch() on the returned Promise.

```
async function fetchUser(id) {
  const res = await fetch(`/users/${id}`);
  const user = await res.json();
  return user; // auto-wrapped in Promise
}
fetchUser(1).then(u => console.log(u));
```

Q94. Explain how promises can be chained.

Each .then() returns a new Promise, allowing chaining. The return value of one .then() is passed to the next. A .catch() anywhere in the chain catches all preceding errors.

```
fetch('/api/user')
  .then(res => res.json())
  .then(user => fetch(`/api/posts/${user.id}`))
  .then(res => res.json())
  .then(posts => console.log(posts))
  .catch(err => console.error(err));
```

Q98. Explain the concept of promises with async/await.

async/await is built on top of Promises. await unwraps a Promise's resolved value. async functions return a Promise. This combination makes async code linear and readable, avoiding .then() chains while keeping non-blocking behaviour.

```

async function main() {
  const res = await fetch('/api/data'); // Promise
  const data = await res.json();        // Promise
  return data;                          // Promise<data>
}
main()
  .then(data => console.log(data))
  .catch(err => console.error(err));

```

Q100. What is the purpose of Promise.all in JavaScript?

Promise.all() takes an array of Promises and returns a single Promise that resolves when ALL input Promises resolve (with an array of results), or rejects immediately if ANY one rejects. Use it to run async operations in parallel.

```

const p1 = fetch('/api/users').then(r => r.json());
const p2 = fetch('/api/posts').then(r => r.json());
const p3 = fetch('/api/comments').then(r => r.json());
const [users, posts, comments] = await Promise.all([p1, p2, p3]);
// All three requests run in parallel

```

Q72. What is the purpose of setTimeout and setInterval?

setTimeout(fn, delay) — executes a function once after the specified delay (ms). **setInterval(fn, interval)** — executes a function repeatedly at the specified interval. Both return an ID that can be used to cancel them with clearTimeout/clearInterval.

```

const t = setTimeout(() => console.log('once'), 1000);
clearTimeout(t); // cancel if needed
const i = setInterval(() => console.log('repeat'), 500);
setTimeout(() => clearInterval(i), 3000); // stop after 3s

```

Q18. Explain event-driven programming in JavaScript.

Event-driven programming is a paradigm where the flow of the program is determined by events — user actions, network responses, timers, etc. JavaScript registers event listeners that are called when specific events occur, rather than executing code top-to-bottom sequentially.

```

document.getElementById('btn')
  .addEventListener('click', function(event) {
    console.log('Button clicked!', event.target);
  });
// Node.js example
emitter.on('data', (chunk) => console.log(chunk));

```

SECTION 5: Arrays & Iteration Methods

Q23. What is the purpose of the map method in JavaScript?

map() creates a new array by calling a function on every element of the original array. It does not mutate the original array and always returns an array of the same length.

```
const nums    = [1, 2, 3, 4];
const doubled = nums.map(n => n * 2);
console.log(doubled); // [2, 4, 6, 8]
console.log(nums);    // [1, 2, 3, 4] – unchanged
```

Q27. What is the purpose of the reduce method in JavaScript?

reduce() executes a reducer function on each element, accumulating a single output value. It takes a callback (accumulator, currentValue) and an optional initial value.

```
const nums = [1, 2, 3, 4, 5];
const total = nums.reduce((acc, n) => acc + n, 0);
console.log(total); // 15
// Flatten array with reduce
[[1,2],[3,4]].reduce((acc, arr) => acc.concat(arr), []);
// [1, 2, 3, 4]
```

Q29. What are the different types of loops in JavaScript?

for — classic index loop. **while** — runs while condition is true. **do...while** — runs at least once. **for...of** — iterates over iterable values (arrays, strings). **for...in** — iterates over object keys. **forEach** — array method, no break support.

```
for (let i = 0; i < 3; i++) {}           // classic
while (condition) {}                   // while
do { } while (condition);               // do-while
for (const item of [1,2,3]) {}          // for-of
for (const key in { a:1, b:2 }) {}      // for-in
[1,2,3].forEach(item => {});            // forEach
```

Q35. What is the purpose of the forEach method in JavaScript?

forEach() executes a provided function once for each array element, in order. It always returns undefined and cannot be stopped with break or return. Use it for side effects (logging, DOM updates), not for transforming data.

```
const fruits = ['apple', 'banana', 'cherry'];
fruits.forEach((fruit, index) => {
  console.log(index + ': ' + fruit);
});
// 0: apple 1: banana 2: cherry
```

Q38. What is the purpose of the splice method in JavaScript arrays?

splice() modifies an array in place by removing, replacing, or inserting elements. It returns an array of removed elements. Arguments: start index, deleteCount, ...items to insert.

```
const arr = ['a', 'b', 'c', 'd'];
arr.splice(1, 2);           // removes 'b','c' → arr = ['a','d']
arr.splice(1, 0, 'x');      // inserts 'x' at index 1
arr.splice(1, 1, 'y');      // replaces index 1 with 'y'
```

Q41. What is the purpose of the split method in JavaScript?

split() is a String method that divides a string into an array of substrings based on a specified separator. It does not mutate the original string.

```
const str = 'Hello World JS';
str.split(' ');           // ['Hello', 'World', 'JS']
str.split('');            // splits into individual chars
str.split(' ', 2);        // ['Hello', 'World'] – limit 2
// Common use: reverse a string
'hello'.split('').reverse().join(''); // 'olleh'
```

Q48. What is the difference between slice and splice in JavaScript arrays?

Feature	slice	splice
Mutates original?	No	Yes
Returns	New array (copy)	Removed elements
Use case	Extract a portion	Add/remove/replace
Arguments	start, end	start, deleteCount, ...items

```
const arr = [1, 2, 3, 4, 5];
arr.slice(1, 3);           // [2, 3] – arr unchanged
arr.splice(1, 2);          // [2, 3] removed – arr = [1,4,5]
```

Q52. What is the purpose of the filter method in JavaScript?

filter() creates a new array containing only the elements that pass the test implemented by the provided callback function. It does not mutate the original array.

```
const nums = [1, 2, 3, 4, 5, 6];
const even = nums.filter(n => n % 2 === 0);
console.log(even); // [2, 4, 6]
const users = [{ age: 17 }, { age: 22 }, { age: 15 }];
const adults = users.filter(u => u.age >= 18);
```

Q58. How can you remove duplicates from an array in JavaScript?

The cleanest way is using a Set, which only stores unique values. Other approaches include filter with indexOf or reduce.

```
const arr    = [1, 2, 2, 3, 3, 4];
// Method 1: Set (cleanest)
const unique = [...new Set(arr)]; // [1, 2, 3, 4]
// Method 2: filter
arr.filter((v, i, a) => a.indexOf(v) === i);
// Method 3: reduce
arr.reduce((acc, v) => acc.includes(v) ? acc : [...acc, v], []);
```

Q59. What is the purpose of the includes method in JavaScript?

`includes()` checks whether an array or string contains a specified value, returning true or false. It uses strict equality (`===`) and correctly handles NaN, unlike `indexOf`.

```
[1, 2, 3].includes(2);      // true
[1, 2, 3].includes(4);      // false
[1, NaN].includes(NaN);     // true (indexOf fails here)
'Hello World'.includes('World'); // true
'Hello World'.includes('world'); // false – case sensitive
```

Q66. What is the purpose of the find method in JavaScript arrays?

`find()` returns the first element in the array that satisfies the provided testing function. Returns undefined if no element is found. Use `findIndex()` to get the index instead.

```
const users = [
  { id: 1, name: 'Alice' },
  { id: 2, name: 'Sanket' },
  { id: 3, name: 'Bob' },
];
const user = users.find(u => u.id === 2);
console.log(user); // { id: 2, name: 'Sanket' }
```

Q70. How do you flatten an array in JavaScript?

Use `Array.flat(depth)` to flatten nested arrays. `flat()` with Infinity flattens all levels. `flatMap()` maps then flattens one level.

```
const nested = [1, [2, [3, [4]]]];
nested.flat();           // [1, 2, [3, [4]]] – 1 level
nested.flat(2);          // [1, 2, 3, [4]]   – 2 levels
nested.flat(Infinity);   // [1, 2, 3, 4]      – all levels
// flatMap: map + flat(1)
[1,2,3].flatMap(x => [x, x*2]); // [1,2,2,4,3,6]
```

Q76. What is the purpose of the every and some methods?

every() — returns true only if ALL elements pass the test. **some()** — returns true if AT LEAST ONE element passes the test. Both short-circuit (stop early).

```
const nums = [2, 4, 6, 8];
nums.every(n => n % 2 === 0); // true - all even
nums.some(n => n > 5);         // true - 6 and 8 pass
nums.every(n => n > 5);         // false - 2 and 4 fail
nums.some(n => n > 10);         // false - none pass
```

Q86. Explain the difference between map, filter, and reduce.

Method	Purpose	Returns
map	Transform each element	New array (same length)
filter	Keep elements matching a condition	New array (shorter or equal)
reduce	Accumulate to a single value	Single value (any type)

```
const nums = [1, 2, 3, 4, 5];
nums.map(n => n * 2);           // [2,4,6,8,10]
nums.filter(n => n > 2);         // [3,4,5]
nums.reduce((acc,n) => acc + n, 0); // 15
```

SECTION 6: DOM & Events

Q13. What is event bubbling and event capturing?

Event Bubbling — event starts at the target element and bubbles up to the root (default behaviour). **Event Capturing** — event starts at the root and travels down to the target (use capture: true). Use stopPropagation() to prevent either.

```
// Bubbling (default)
el.addEventListener('click', handler);
// Capturing phase
el.addEventListener('click', handler, true);
// Stop propagation
el.addEventListener('click', e => e.stopPropagation());
```

Q31. Explain the concept of event propagation in JavaScript.

Event propagation describes how events travel through the DOM in 3 phases: **Capture phase** — from window down to target. **Target phase** — event reaches the target element. **Bubble phase** — event bubbles back up to window. addEventListener's third argument (useCapture) controls which phase the listener fires on.

Q32. What are the different ways to manipulate the DOM in JavaScript?

Common DOM manipulation methods:

```
// Selecting
document.getElementById('id');
document.querySelector('.class');
document.querySelectorAll('p');
// Creating & inserting
const el = document.createElement('div');
parent.appendChild(el);
parent.insertBefore(el, referenceNode);
// Modifying
el.textContent = 'Hello';
el.innerHTML = '<b>Bold</b>';
el.setAttribute('class', 'active');
el.style.color = 'red';
// Removing
el.remove();
parent.removeChild(el);
```

Q43. Explain the concept of the event delegation pattern.

Event delegation is attaching a single event listener to a parent element to handle events from multiple child elements — leveraging event bubbling. Benefits: fewer event listeners (better memory), works for dynamically added children.

```
// Instead of:
document.querySelectorAll('li').forEach(li =>
  li.addEventListener('click', handler));
// Use delegation:
document.querySelector('ul')
  .addEventListener('click', e => {
    if (e.target.matches('li')) {
      console.log(e.target.textContent);
    }
  });
```

Q75. How do you handle data binding in JavaScript?

Data binding syncs data between the model (JS) and the view (DOM). **One-way binding** — data flows from model to view (innerHTML, textContent). **Two-way binding** — view updates reflect back to model (input event listeners). Frameworks like React/Vue automate this.

```
// One-way
document.getElementById('name').textContent = user.name;
// Two-way (manual)
const input = document.getElementById('input');
input.value = data.text;
input.addEventListener('input', e => {
  data.text = e.target.value;
});
```

SECTION 7: ES6+ Modern JavaScript

Q51. What are arrow functions, and how are they different from regular functions?

Arrow functions are a concise function syntax introduced in ES6. Key differences from regular functions:

Feature	Regular Function	Arrow Function
Syntax	function() {}	() => {}
'this' binding	Dynamic (caller)	Lexical (outer scope)
arguments object	Yes	No
Used as constructor	Yes	No
Hoisted	Yes (declaration)	No

```
const add = (a, b) => a + b;      // implicit return
const greet = name => `Hi ${name}`; // single param
const obj = {
  val: 10,
  regular() { return this.val; }, // this = obj
  arrow: () => this.val,          // this = outer
};
```

Q53. Explain the concept of destructuring in JavaScript.

Destructuring is an ES6 syntax that allows unpacking values from arrays or properties from objects into distinct variables.

```
// Array destructuring
const [a, b, c] = [1, 2, 3];
const [first, , third] = [10, 20, 30]; // skip 20
// Object destructuring
const { name, age } = { name: 'Sanket', age: 23 };
// Rename + default
const { name: n = 'Guest', role = 'user' } = user;
// Function params
function greet({ name, age }) { return `${name}, ${age}`; }
```

Q61. What are template literals, and how are they used?

Template literals are strings enclosed in backticks (`) that support multi-line strings, embedded expressions `\${...}`, and tagged templates. They make string construction cleaner and more readable.

```
const name = 'Sanket';
const age = 23;
// Expression interpolation
console.log(`Hello, ${name}! You are ${age} years old.`);
// Multi-line (no \n needed)
const msg = `Line 1
Line 2`;
// Expression
console.log(`2 + 2 = ${2 + 2}`); // '2 + 2 = 4'
```

Q60. How do you handle default values in JavaScript functions?

ES6 default parameters allow you to set fallback values for function arguments when they are undefined or not passed.

```
function greet(name = 'Guest', role = 'user') {  
  return `Hello ${name}, role: ${role}`;  
}  
greet();           // 'Hello Guest, role: user'  
greet('Sanket');   // 'Hello Sanket, role: user'  
// With destructuring defaults  
function setup({ port = 3000, host = 'localhost' } = {}) {  
  return `${host}:${port}`;  
}
```

Q62. Explain the concept of modules in JavaScript.

Modules allow you to split code into reusable files. ES6 introduced native modules with import/export. Each module has its own scope — variables are not global by default. Modules are loaded once and cached.

```
// math.js – named exports  
export const PI = 3.14;  
export function add(a, b) { return a + b; }  
// Default export  
export default function multiply(a, b) { return a * b; }  
// main.js – importing  
import multiply, { PI, add } from './math.js';
```

Q67. How does import and export work in JavaScript?

Named exports — export multiple values, imported with exact names in {}. **Default export** — one per module, imported without {}. **Re-export** — pass-through with export ... from.

```
// Named  
export const x = 1;  
import { x } from './module';  
// Default  
export default class MyClass {}  
import MyClass from './module';  
// Alias  
import { x as myX } from './module';  
// Import all  
import * as utils from './utils';
```

Q71. What are IIFE (Immediately Invoked Function Expressions)?

An IIFE is a function that is defined and immediately executed. It creates its own scope, preventing variables from polluting the global scope. Widely used before ES6 modules.

```
(function() {
  const secret = 'I am private';
  console.log('IIFE runs immediately');
})();
// Arrow function IIFE
(() => {
  console.log('Arrow IIFE');
})();
// With return value
const result = (function() { return 42; })();
```

Q39. What is a generator function in JavaScript?

A generator function (function*) can pause execution and resume later, yielding multiple values lazily. It returns an iterator object. Use yield to produce values one at a time.

```
function* range(start, end) {
  for (let i = start; i <= end; i++) {
    yield i;
  }
}
const gen = range(1, 3);
gen.next(); // { value: 1, done: false }
gen.next(); // { value: 2, done: false }
gen.next(); // { value: 3, done: false }
gen.next(); // { value: undefined, done: true }
```

SECTION 8: Objects, Patterns & Advanced Concepts

Q20. What is the difference between a deep copy and a shallow copy?

Shallow copy — copies only the top-level properties. Nested objects still share the same reference. **Deep copy** — recursively copies all nested objects/arrays; fully independent copy.

```
const obj = { a: 1, b: { c: 2 } };
// Shallow – nested still shared
const shallow = { ...obj };
shallow.b.c = 99;
console.log(obj.b.c); // 99 – affected!
// Deep copy
const deep = JSON.parse(JSON.stringify(obj));
deep.b.c = 99;
console.log(obj.b.c); // 2 – unaffected
```

Q42. What is the difference between a deep clone and a shallow clone?

Same concept as deep/shallow copy. **Shallow clone** methods: spread {...obj}, Object.assign(). **Deep clone** methods: JSON.parse(JSON.stringify()) (simple but loses functions/undefined/Date), structuredClone() (modern, handles more types), or libraries like Lodash _.cloneDeep().

```
// Shallow
const s1 = Object.assign({}, obj);
const s2 = { ...obj };
// Deep
const d1 = JSON.parse(JSON.stringify(obj)); // simple
const d2 = structuredClone(obj);           // modern (2022+)
```

Q33. What is the purpose of localStorage and sessionStorage?

Both are Web Storage APIs for storing key-value pairs in the browser. **localStorage** — persists until explicitly cleared; survives page reloads and browser restarts. **sessionStorage** — cleared when the browser tab is closed; scoped to the session.

```
// localStorage
localStorage.setItem('token', 'abc123');
localStorage.getItem('token'); // 'abc123'
localStorage.removeItem('token');
// sessionStorage
sessionStorage.setItem('step', '2');
sessionStorage.getItem('step'); // '2'
// Cleared when tab closes
```

Q65. Explain the concept of immutability in JavaScript.

Immutability means a value cannot be changed after it is created. Primitives are immutable by nature. For objects, immutability must be enforced manually using `Object.freeze()`, `const` (prevents re-assignment, not mutation), or libraries like `Immer`.

```
// Object.freeze – prevents modification
const config = Object.freeze({ apiUrl: '/api', timeout: 5000 });
config.apiUrl = '/new'; // silently fails (strict: throws)
console.log(config.apiUrl); // '/api' – unchanged
// Immutable update pattern
const updated = { ...config, timeout: 3000 }; // new object
```

Q68. What is a Proxy object in JavaScript?

A Proxy wraps an object and intercepts fundamental operations like `get`, `set`, `delete` using 'traps'. It enables custom behaviour for property access, validation, logging, etc.

```
const handler = {
  get(target, key) {
    return key in target ? target[key] : `No prop: ${key}`;
  },
  set(target, key, value) {
    if (typeof value !== 'number') throw new TypeError();
    target[key] = value;
    return true;
  }
};
const proxy = new Proxy({}, handler);
proxy.age = 23; // OK
proxy.name; // 'No prop: name'
```

Q74. What is a WeakMap, and how is it different from a Map?

Feature	Map	WeakMap
Key types	Any type	Objects only
Iterable	Yes	No
Size property	Yes (.size)	No
GC of keys	Prevents GC	Keys are weakly held (GC-able)
Use case	General key-value store	Private data, caching

```
const wm = new WeakMap();
let obj = {};
wm.set(obj, 'private data');
wm.get(obj); // 'private data'
obj = null; // obj can now be garbage collected
```

Q78. How do you remove properties from an object in JavaScript?

Use the delete operator to remove a property. For immutable patterns, use object destructuring with rest to create a new object without the property.

```
const user = { name: 'Sanket', age: 23, role: 'dev' };
// Mutate – delete operator
delete user.age;
console.log(user); // { name: 'Sanket', role: 'dev' }
// Immutable – destructure + rest
const { age, ...rest } = user;
console.log(rest); // { name: 'Sanket', role: 'dev' }
```

Q79. What is the purpose of Object.freeze?

Object.freeze() makes an object immutable — prevents adding, removing, or modifying properties. It is shallow: nested objects are NOT frozen. Returns the same frozen object.

```
const settings = Object.freeze({
  theme: 'dark',
  lang: 'en',
});
settings.theme = 'light'; // silently ignored
settings.newProp = 'x'; // silently ignored
console.log(settings.theme); // 'dark'
// Nested objects are NOT frozen
const obj = Object.freeze({ nested: { val: 1 } });
obj.nested.val = 99; // Works! (shallow freeze)
```

Q80. How do you check if an object is empty in JavaScript?

The most reliable way is Object.keys(obj).length === 0, which checks own enumerable properties. Combine with a typeof check for robustness.

```
const isEmpty = obj =>
  obj !== null &&
  typeof obj === 'object' &&
  Object.keys(obj).length === 0;
isEmpty({}); // true
isEmpty({ a: 1 }); // false
isEmpty([]); // true (arrays are objects)
// Also:
JSON.stringify(obj) === '{}'; // quick but less reliable
```

Q85. What is the purpose of `Object.assign`?

`Object.assign(target, ...sources)` copies all enumerable own properties from one or more source objects into a target object. It mutates and returns the target. Used for shallow merging and cloning.

```
const defaults = { color: 'blue', size: 'M' };
const custom = { size: 'L', weight: 'light' };
const result = Object.assign({}, defaults, custom);
// { color: 'blue', size: 'L', weight: 'light' }
// Shallow clone
const clone = Object.assign({}, original);
```

Q99. How do you use `Object.keys`, `Object.values`, and `Object.entries`?

`Object.keys()` — returns array of own enumerable property names. **`Object.values()`** — returns array of values. **`Object.entries()`** — returns array of [key, value] pairs. All work on own enumerable properties only.

```
const user = { name: 'Sanket', age: 23, role: 'dev' };
Object.keys(user); // ['name', 'age', 'role']
Object.values(user); // ['Sanket', 23, 'dev']
Object.entries(user); // [['name', 'Sanket'], ['age', 23], ['role', 'dev']]
// Iterate with entries
Object.entries(user).forEach(([key, val]) => {
  console.log(`${key}: ${val}`);
});
```

SECTION 9: Performance, Patterns & Miscellaneous

Q54. What is callback hell, and how can it be avoided?

Callback hell (pyramid of doom) occurs when callbacks are nested inside callbacks, creating deeply indented, hard-to-read code. Avoid it with: **Promises**, **`async/await`**, or modularizing callbacks into named functions.

```
// Callback hell
getData(function(a) {
  getMore(a, function(b) {
    getEven(b, function(c) {
      console.log(c); // deeply nested!
    });
  });
});
// async/await – clean
const a = await getData();
const b = await getMore(a);
const c = await getEven(b);
console.log(c);
```

Q55. What are promises and how do they work in JavaScript?

A Promise represents the eventual result of an async operation. It is created with `new Promise((resolve, reject) => {})`. Methods: `.then()` for success, `.catch()` for error, `.finally()` for cleanup — always runs. Promises are always asynchronous even if resolved immediately.

```
function delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
delay(1000)
  .then(() => console.log('1 second passed'))
  .catch(err => console.error(err))
  .finally(() => console.log('Done'));
```

Q57. What is JSON, and how is it used in JavaScript?

JSON (JavaScript Object Notation) is a lightweight data-interchange format. In JavaScript, `JSON.stringify()` converts objects to JSON strings, and `JSON.parse()` parses JSON strings back to objects. It only supports strings, numbers, booleans, null, arrays, and objects — not functions, undefined, or Symbol.

```
const user = { name: 'Sanket', age: 23 };
// Object → JSON string
const json = JSON.stringify(user);
// '{"name":"Sanket","age":23}'
// JSON string → Object
const parsed = JSON.parse(json);
parsed.name; // 'Sanket'
// Pretty print
JSON.stringify(user, null, 2);
```

Q63. What is CORS, and why is it important?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts HTTP requests made to a different origin (domain, port, protocol) than the one that served the page. The server must include Access-Control-Allow-Origin headers to permit cross-origin requests. Without it, browsers block the response.

```
// Server must respond with:
// Access-Control-Allow-Origin: https://yoursite.com
// Preflight (OPTIONS) request sent for:
// - Non-simple methods (PUT, DELETE, PATCH)
// - Custom headers
// - Content-Type: application/json
```

Q64. What is the fetch API and how does it work?

fetch() is a modern, Promise-based API for making HTTP requests. It replaces XMLHttpRequest. Returns a Promise that resolves to a Response object. You must call .json(), .text(), or .blob() on it to get the actual data — these are also Promises.

```
fetch('https://api.example.com/users', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ name: 'Sanket' }),
})
  .then(res => {
    if (!res.ok) throw new Error(res.status);
    return res.json();
  })
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Q73. What is a polyfill in JavaScript?

A polyfill is code that implements a feature on browsers that don't natively support it. It 'fills in' the gap by checking if the feature exists and providing a fallback implementation. Common polyfills: Promise, Array.includes, Object.assign, fetch.

```
// Polyfill for Array.prototype.includes
if (!Array.prototype.includes) {
  Array.prototype.includes = function(value) {
    return this.indexOf(value) !== -1;
  };
}
```

Q81. Explain the concept of throttling in JavaScript.

Throttling limits the execution of a function to at most once per specified time interval, regardless of how many times it is triggered. Used for scroll, resize, and mousemove events to prevent performance issues.

```
function throttle(fn, limit) {
  let lastCall = 0;
  return function(...args) {
    const now = Date.now();
    if (now - lastCall >= limit) {
      lastCall = now;
      return fn(...args);
    }
  };
}
const throttledScroll = throttle(() => console.log('scroll'), 200);
window.addEventListener('scroll', throttledScroll);
```

Q82. What is a WeakSet in JavaScript?

A WeakSet is a collection of objects (no primitives) where references are weakly held — if no other reference to the object exists, it can be garbage collected. WeakSets are not iterable and have no size property. Use case: tracking objects without preventing GC.

```
const ws = new WeakSet();
let obj1 = { id: 1 };
let obj2 = { id: 2 };
ws.add(obj1);
ws.has(obj1); // true
ws.has(obj2); // false
obj1 = null; // obj1 can now be garbage collected
```

Q84. What is debouncing and how is it different from throttling?

Debouncing — delays function execution until after a specified time has passed since the last invocation. Resets the timer on each call. Use for: search input, resize end. **Throttling** — executes at most once per interval, regardless of calls.

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
const search = debounce(query => fetchResults(query), 300);
input.addEventListener('input', e => search(e.target.value));
// Only fires 300ms after user stops typing
```

Q90. What are JavaScript decorators?

Decorators are a proposal (Stage 3) that allow you to annotate and modify classes, methods, and properties using `@decorator` syntax. They are functions that receive the target and return a modified version. Currently used widely via TypeScript and Babel.

```
// Method decorator example (TypeScript/Babel)
function readonly(target, key, descriptor) {
  descriptor.writable = false;
  return descriptor;
}
class Person {
  @readonly
  greet() { return 'Hello'; }
}
// Class decorator
function sealed(constructor) {
  Object.seal(constructor);
  Object.seal(constructor.prototype);
}
@sealed class API {}
```

Q91. How do you optimize JavaScript code for performance?

Key optimizations:

1. Minimize DOM access — cache references, batch updates. **2. Debounce/throttle** high-frequency events. **3. Use efficient loops** — cache array length. **4. Avoid memory leaks** — clear timers and event listeners. **5. Lazy loading** — load resources on demand. **6. Use Web Workers** for heavy computation. **7. Code splitting** — ship only what's needed.

```
// Cache DOM reference
const list = document.getElementById('list'); // once
// Cache array length
for (let i = 0, len = arr.length; i < len; i++) {}
// Batch DOM updates with DocumentFragment
const frag = document.createDocumentFragment();
items.forEach(item => frag.appendChild(createEl(item)));
list.appendChild(frag); // single reflow
```

Q92. What is a service worker in JavaScript?

A service worker is a script that runs in the background, separate from the main thread, acting as a proxy between the browser and the network. It enables: offline support (caching), push notifications, and background sync. It is part of Progressive Web Apps (PWA).

```
// Register service worker
if ('serviceWorker' in navigator) {
  navigator.serviceWorker
    .register('/sw.js')
    .then(reg => console.log('SW registered', reg));
}
// sw.js – cache on install
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open('v1').then(c => c.addAll(['/','index.html']))
  );
});
```

Q93. How do you use try...catch for error handling in JavaScript?

try — wraps code that might throw. catch(err) — handles the error; err has message, name, stack. finally — always runs, used for cleanup. You can throw custom errors with new Error() or custom classes.

```
try {
  const data = JSON.parse(invalidJson);
  riskyOperation(data);
} catch (err) {
  if (err instanceof SyntaxError) {
    console.error('JSON parse error:', err.message);
  } else {
    throw err; // re-throw unknown errors
  }
} finally {
  console.log('Cleanup done');
}
```

Q96. How can you reverse a string in JavaScript?

The classic one-liner: split into characters → reverse the array → join back. Multiple approaches exist:

```
const str = 'Sanket';
// Method 1: split-reverse-join
str.split('').reverse().join(''); // 'teknaS'
// Method 2: spread + reverse + join
[...str].reverse().join(''); // handles Unicode better
// Method 3: for loop
let rev = '';
for (let i = str.length - 1; i >= 0; i--) rev += str[i];
// Method 4: reduce
str.split('').reduce((acc, ch) => ch + acc, '');
```

Q97. What is a Set in JavaScript?

A Set is a collection of unique values of any type. Duplicates are automatically ignored. Unlike arrays, Sets don't have indexes. Common use: removing duplicates, membership testing.

```
const set = new Set([1, 2, 2, 3, 3]);
console.log(set); // Set {1, 2, 3}
set.add(4);
set.has(2); // true
set.size; // 4
set.delete(1);
// Convert to array
[...set]; // [2, 3, 4]
Array.from(set); // [2, 3, 4]
// Remove duplicates from array
[...new Set([1,1,2,3,3])]; // [1, 2, 3]
```